Handoff 52: Visual redesign (one pull request)

Date: 2026-11-09 (America/Toronto)
Written by: Claude Code (implementer), in a separate git worktree
For: the human maintainer, and ChatGPT or Claude chat (reviewer) without repository access
Status: **in_review**: one pull request, not merged, not deployed. **No migration, no new dependency, no backend change, no Content-Security-Policy change.**

> **How to use this file:** upload it with `docs/design/prototype/index.html` (the design source) and `docs/design/art-style.md`. The pull request diff is the source of truth (`build.md` section 14).

Starter prompt for the chat assistant

```
You are reviewing the visual redesign of Python Academy. The owner's prototype
(docs/design/prototype/index.html) is now the real site: one stylesheet with design tokens, 16
lesson pictures keyed by lesson id with a plain fallback, a five-stop progress path and card-type
tabs on the lesson page, worked-example markers, a challenge "quest ticket" built from the check's
visible cases plus locked hidden ones, and new signed-out and student home pages. Check that hidden
expected output can never reach or appear on the page, that the account-rule wording is true, that
the home page playground cannot load Python, and that nothing in the lesson text changed.
```

Run plan details

Field	Value
Request	The owner's "Build the visual redesign as ONE pull request" prompt, 2026-11-09
Base	`main`, at `f37387d` (after #59)
Branch and worktree	`ui/visual-redesign`, `../python-academy-redesign`; the main folder was not switched
Merge order	Any. No other PR is open

1. What was built

Request item	Where	Notes
Prototype copied into the repo	`docs/design/prototype/index.html`	Byte-for-byte copy of the owner's revised file
1. Tokens and one stylesheet	`frontend/app/style.css` (rewritten)	Prototype palette as `:root` variables; serif headings, mono code; card-type colours: discover amber, example moss, predict slate, try plum, challenge clay. The topographic texture is `public/topo.svg` (the page policy's `img-src 'self'` blocks the prototype's `data:` URL)
2. 16 lesson pictures and fallback	`frontend/app/lesson-art.tsx`	Generated from the prototype's `<symbol>` elements, then kept by hand. `LESSON_ART` maps each lesson id to its picture and a written description. Unknown ids get a plain picture. The castle `World()` is gone
3. Lesson page	`frontend/app/adventure.tsx`	Lesson title as the heading with its picture; five-stop path with icons; card-type tab; pictures in the unit list; worked-example markers; predict answer tiles; quest ticket
4. Home, signed out	`frontend/app/home.tsx` (`Landing`)	Prototype copy, minus the switcher bar, with the account line and parents section checked (section 3)
4. Home, student	`home.tsx` (`StudentHome`)	Continue card with the recommended lesson's picture, "Your path" unit strips with a dot per lesson, "Coming later" cards
4. Course page and catalog rows	`frontend/app/course.tsx`, `catalog.tsx`	Side box, unit cards, rows with each lesson's picture and state
Header and footer	`site-header.tsx`, `page-shell.tsx`	Prototype header; "Sign in" and "Try a lesson" only when signed out
5. Art guide	`docs/design/art-style.md`	Frame, tints by unit, shapes, words in pictures, accessibility, how to add one

How the quest ticket gets its cases (and why hidden output can't leak)

On the challenge card of a lesson with a browser check, the page fetches the same plan a run already uses (`GET /api/checks/<lesson>`). The API leaves hidden cases' expected output out (`browser_checks.plan`). The ticket lists each **visible** case's description and what it should print (or return); **hidden cases appear only as a count with a lock**, and their descriptions aren't shown either. Even if a plan ever carried a hidden case's output, the ticket would not display it; a test checks that with a doctored plan. After a check, each visible case shows passed or not, from the `cases` statuses the API already returns, in words for screen

readers as well as colour. The run now reuses the plan the ticket fetched, so there is no extra request. Unit 1's challenges have no browser check, so they show no ticket.

Worked-example markers

On "See it work", a number beside each `print()` line matches the output line it made. They appear only when the pairing is certain: top-level code with no blocks, one `print()` per line, no `end=` or `sep=`, and as many prints as output lines. Otherwise (loops, `if`) there are no markers. The numbers are CSS generated content, so the code and output text stay exactly as written.

2. Screenshots

Desktop (1280 px) and phone (390 px), from the built app with the recorded API answers:

py Python Academy Courses Try a lesson Sign in

Your first Python program is a story.

Write a few lines, run them, see what happens. Each lesson ends with a small challenge you solve yourself.

Try a lesson, no account → Sign in with Google

Learners 13 and up sign in through a parent's invitation. Anyone over 25 can start a family account. Accounts for learners 16 and up on their own are planned, not open yet.

Preview count_the_treasure.py unit 2 · lesson 1

```
# share a chest of coins
coins = 17
friends = 5
print(coins // friends)
print(coins % friends)
```

OUTPUT

```
3
2
```

Change the numbers. `//` is each friend's whole share, `%` is what's left over. In a lesson you type these lines yourself.

Every lesson goes in the same order.

So you always know what is being asked of you.

Discover
One idea, in a few sentences.

Worked example
Code you read line by line, with the output beside it.

Predict
Guess what it prints before it runs. Guess wrong and you get the reason.

Try it
Change the example and see what moves.

Challenge
Write the program yourself. Hidden test cases catch copied answers.

When it breaks, you are told why.

The page marks the line, says what Python couldn't do, and often guesses the name you meant. Python's own message is a click away.

Stuck? Hints come in steps: a nudge, a bigger hint, then the approach. There is no button that hands you the answer.

hero.py 3 lines

```
1  hero = "Kai"
2  tool = "map"
▶ 3  print(heor)
```

PYTHON STOPPED · LINE 3

Python does not know `heor`

You used a name before giving it a value, or spelled it two ways. Did you mean `hero`?

A nudge A bigger hint The approach

Hints appear here, one at a time.

Python Foundations: three units open, nine on the way.

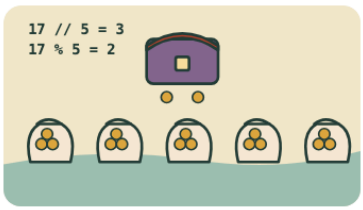
Each unit takes the same adventure a little further.



UNIT 1 · 5 lessons

Build Your Own Adventure

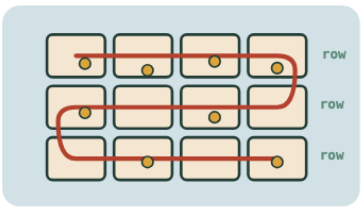
Print, variables, input and choices, ending in a small game that is yours.



UNIT 2 · 6 lessons

Numbers and decisions

Arithmetic, comparisons, and, or, not and elif, through a riddle keeper's game.



UNIT 3 · 5 lessons

Repeat and explore

while and for loops that always finish, and a loop inside a loop.

Unit 4 · Keep track of a world
Lists, dictionaries, sets

Unit 5 · Organize your code
Functions and scope

Units 6 to 12
Errors, libraries, tests, files, patterns in text, classes, a final project

After Python: Hackathon Ready, Git and GitHub, web, JavaScript and SQL, all marked *coming soon*.

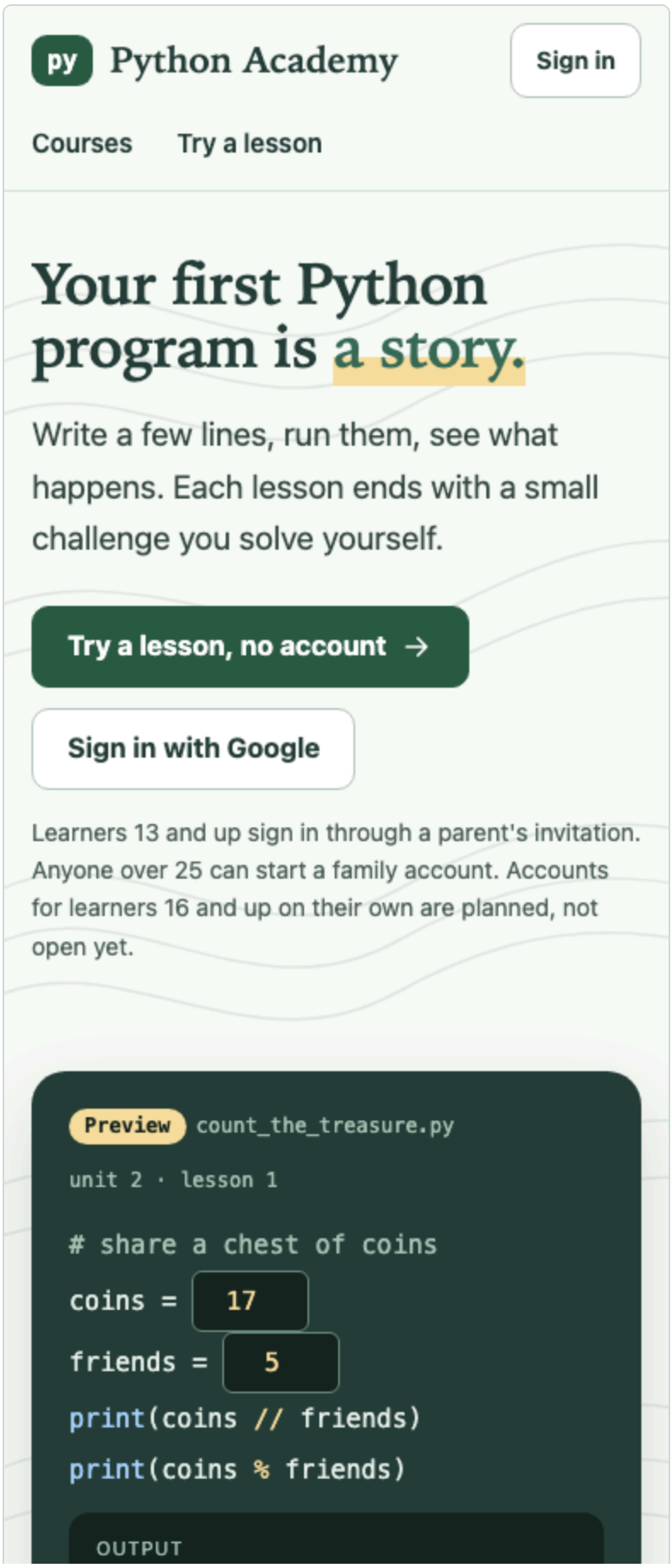
For parents

A parent over 25 creates a family account and invites each learner by Google email. Try any lesson first; nothing from a preview is saved.

Try a lesson as a parent

- ✓ Learner code runs in the browser. It is never run on our servers.
- ✓ You see their chosen goal, which lessons are finished, and how many of their attempts passed.
- ✓ Invitations expire after seven days and work only for the invited email.
- ✓ Sign-in is through Google. We never see a password.

Python Academy is a one-person project, still early. Three units are open and the rest are planned. Checks are practice, not a certificate.


py
Python Academy
Sign in
Courses
Try a lesson
Your first Python program is a story.
Write a few lines, run them, see what happens. Each lesson ends with a small challenge you solve yourself.
Try a lesson, no account →
Sign in with Google
Learners 13 and up sign in through a parent's invitation. Anyone over 25 can start a family account. Accounts for learners 16 and up on their own are planned, not open yet.
Preview
count_the_treasure.py
unit 2 · lesson 1
share a chest of coins
coins = 17
friends = 5
print(coins // friends)
print(coins % friends)
OUTPUT

```
3
2
```

Change the numbers. `//` is each friend's whole share, `%` is what's left over. In a lesson you type these lines yourself.

Every lesson goes in the same order.

So you always know what is being asked of you.

Discover

One idea, in a few sentences.

Worked example

Code you read line by line, with the output beside it.

Predict

Guess what it prints before it runs. Guess wrong and you get the reason.

Try it

Change the example and see what moves.

Challenge

Write the program yourself. Hidden test cases catch copied answers.

When it breaks, you are told why.

The page marks the line, says what Python couldn't do, and often guesses the name you meant. Python's own message is a click away.

Stuck? Hints come in steps: a nudge, a bigger hint, then the approach. There is no button that hands you the answer.

hero.py — 3 lines

```
1  hero = "Kai"
2  tool = "map"
▶ 3  print(heor)
```

PYTHON STOPPED · LINE 3

Python does not know `heor`

You used a name before giving it a value, or spelled it two ways. Did you mean `hero`?

A nudge | A bigger hint | The approach

Hints appear here, one at a time.

Python Foundations: three units open, nine on the way.

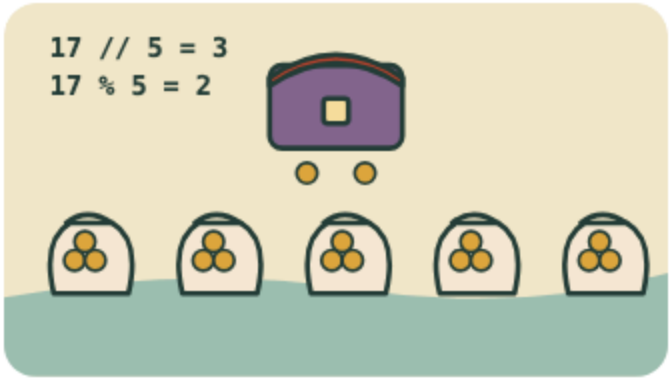
Each unit takes the same adventure a little further.



UNIT 1 5 lessons

Build Your Own Adventure

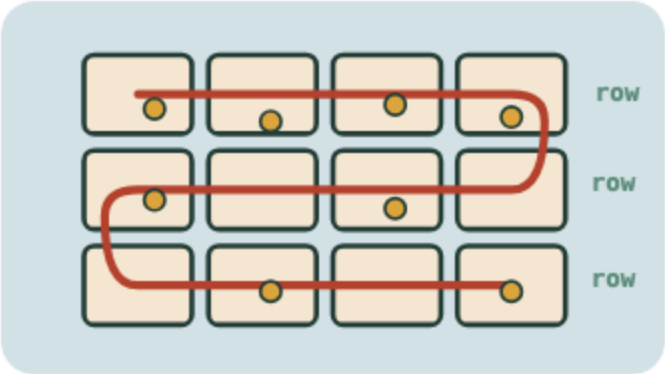
Print, variables, input and choices, ending in a small game that is yours.



UNIT 2 6 lessons

Numbers and decisions

Arithmetic, comparisons, and, or, not and elif, through a riddle keeper's game.



UNIT 3 5 lessons

Repeat and explore

while and for loops that always finish, and a loop inside a loop.

Unit 4 · Keep track of a world
Lists, dictionaries, sets

Unit 5 · Organize your code
Functions and scope

Units 6 to 12
Errors, libraries, tests, files, patterns in text, classes, a final project

After Python: Hackathon Ready, Git and GitHub, web, JavaScript and SQL, all marked *coming soon*.

For parents

A parent over 25 creates a family account and invites each learner by Google email. Try any lesson first; nothing from a preview is saved.

Try a lesson as a parent

- ✓ Learner code runs in the browser. It is never run on our servers.
- ✓ You see their chosen goal, which lessons are finished, and how many of their attempts passed.
- ✓ Invitations expire after seven days and work only for the invited email.
- ✓ Sign-in is through Google. We never see a password.

Python Academy is a one-person project, still early. Three units are open and the rest are planned. Checks are practice, not a certificate.

py Python Academy Courses Sign out

Welcome back. Keep building.

next up · unit 3, Repeat and explore

Keep guessing

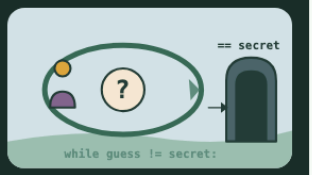
You finished Numbers and decisions, so loops come next on your Learn to code path.

Continue, card 1 of 5 → See the whole unit

1100 XP
11 lessons done

11
badges

Goal: **Learn to code from scratch** · change



Your path in Python Foundations RECOMMENDED

11 of 16 lessons complete



1 · Build Your Own Adventure

5 of 5 lessons done

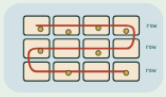
Review



2 · Numbers and decisions

6 of 6 lessons done

Review



3 · Repeat and explore

0 of 5 lessons done

Continue

Coming later

COMING SOON
Hackathon Ready
Needs Python Foundations units 1–7.

COMING SOON
Git and GitHub Essentials
Needs Python Foundations unit 1.

COMING SOON
Web Foundations: HTML and CSS
Needs Python Foundations units 1–3.

COMING SOON
JavaScript and the DOM
Needs Python Foundations units 1–5 and Web Foundations: HTML and CSS.

COMING SOON
Data and SQL
Needs Python Foundations units 1–4.

COMING SOON
Data Investigation with Python
Needs Python Foundations units 1–9 and Data and SQL.

COMING SOON
Computer Science Foundations
Needs Python Foundations units 1–5.

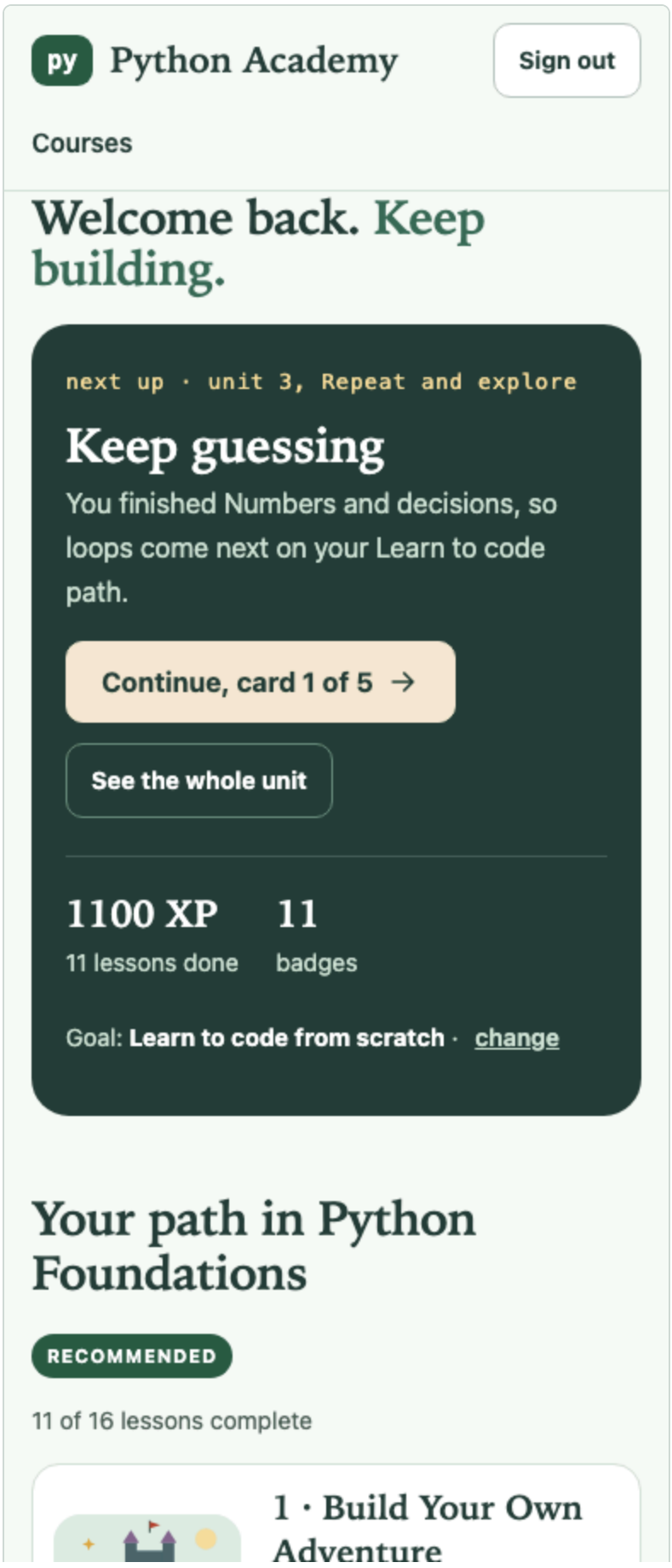
COMING SOON
Node.js and APIs
Needs JavaScript and the DOM and Data and SQL.

COMING SOON
Full-Stack Builder
Needs Git and GitHub Essentials and Node.js and APIs.

COMING SOON
Engineering Depth
Needs Python Foundations and Computer Science Foundations.

▸ **Your account history**

Built for curious minds. Designed for room to grow.


py
Python Academy
Sign out
Courses
Welcome back. Keep building.
next up · unit 3, Repeat and explore
Keep guessing
You finished Numbers and decisions, so loops come next on your Learn to code path.
Continue, card 1 of 5 →
See the whole unit
1100 XP
11 lessons done
11
badges
Goal: Learn to code from scratch · change
Your path in Python Foundations
RECOMMENDED
11 of 16 lessons complete
1 · Build Your Own Adventure



5 of 5 lessons done

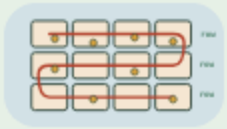
Review



2 · Numbers and decisions

6 of 6 lessons done

Review



3 · Repeat and explore

0 of 5 lessons done

Continue

Coming later

COMING SOON

Hackathon Ready

Needs Python Foundations units 1–7.

COMING SOON

Git and GitHub Essentials

Needs Python Foundations unit 1.

COMING SOON

Web Foundations: HTML and CSS

Needs Python Foundations units 1–3.

COMING SOON

JavaScript and the DOM

Needs Python Foundations units 1–5 and Web Foundations: HTML and CSS.

COMING SOON

Data and SQL

Needs Python Foundations units 1–4.

COMING SOON

Data Investigation with Python

Needs Python Foundations units 1–9 and Data and SQL.

COMING SOON

Computer Science Foundations

Needs Python Foundations units 1–5.

COMING SOON

Node.js and APIs

Needs JavaScript and the DOM and Data and SQL.

COMING SOON

Full-Stack Builder

Needs Git and GitHub Essentials and Node.js and APIs.

COMING SOON

Engineering Depth

Needs Python Foundations and Computer Science Foundations.

▶ **Your account history**

Built for curious minds. Designed for room to grow.

py Python Academy Courses

Sign out

Courses › Python Foundations

AVAILABLE

3 of 12 units open

16 lessons ready

11 of 16 available lessons complete

Continue: Keep guessing →

Builds on nothing. Start here.

Python Foundations

Write your first programs, then grow them into tested, organized projects.

UNIT 1

COMPLETE

Build Your Own Adventure

Output, variables, input, and choices in one connected story.



Open the story

`Output · print()`

Story spark badge

✓ Done



Create your hero

`Variables · strings`

Character creator badge

✓ Done



Meet the player

`Interaction · input()`

Conversation starter badge

✓ Done



Choose a path

`Decisions · if / else`

Path finder badge

✓ Done



Launch your adventure

`Combine your skills`

Adventure maker badge

✓ Done

UNIT 2

COMPLETE

Numbers and decisions

Arithmetic, whole numbers and decimals, numbers from input(), comparisons, Boolean logic and elif.

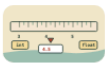


Count the treasure

`Arithmetic · + − * / // %`

Treasure counter badge

✓ Done



Average the scores

`Numbers · int and float`

Score keeper badge

✓ Done



Score the riddle

`Conversion · int(input())`

Riddle scorer badge

✓ Done



Too high or too low?

`Comparisons · < > <= >= == !=`

Riddle judge badge

✓ Done

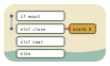


Close enough?

`Logic · and, or, not`

Close call badge

✓ Done



The riddle keeper's verdict

`Decisions · elif`

Fair verdict badge

✓ Done

UNIT 3

Repeat and explore

0 OF 5

while and for loops that always finish, running totals, and loops inside loops.

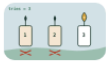


Keep guessing

`Loops · while`

Patient guesser badge

Up next

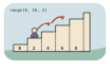


Three tries

`Loops · stopping on time`

Loop closer badge

Not started

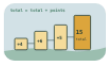


Step by step

`Loops · for and range()`

Step counter badge

Not started

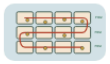


Keep the score

`Loops · running totals and counters`

Tally keeper badge

Not started



Rooms on the map

`Loops · a loop inside a loop`

Map reader badge

Not started

UNIT 4

Keep track of a world

COMING SOON

Lists, dictionaries, and sets.

UNIT 5

Organize your code

COMING SOON

Functions, parameters, return values, and scope.

UNIT 6

Recover and investigate

COMING SOON

Tracebacks, debugging, and exceptions.

UNIT 7

Libraries and modules

COMING SOON

Importing and using the standard library.

UNIT 8

Trust your changes

COMING SOON

Tests, boundaries, and refactoring.

UNIT 9

Save and load

COMING SOON

Text files, JSON, and CSV.

UNIT 10

Patterns in text

COMING SOON

String methods and regular expressions.

UNIT 11

COMING SOON

Model a larger program

Classes and composition.

UNIT 12

COMING SOON

Final project

Plan, build, test, and explain a project of your own.

Built for curious minds. Designed for room to grow.

py Python Academy Courses

Courses › Python Foundations › Unit 2: Numbers and decisions

Content preview Jump to any lesson or card. Practice and badges stay in this session; they are not saved. Return to your account

unit 2
Numbers and decisions

0 of 6 lessons complete

- **Count the treasure** — Arithmetic · + - * / // %
- Average the scores — Numbers · int and float
- Score the riddle — Conversion · int(input())
- Too high or too low? — Comparisons · < > <= >= == !=
- Close enough? — Logic · and, or, not
- The riddle keeper's verdict — Decisions · elif

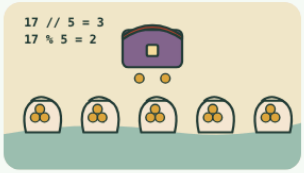
All units in Python Foundations

unit 2 · lesson 1 of 6

Count the treasure

`Arithmetic · + - * / // %` Quest 1 of 6 · Treasure counter

0 XP 0 of 6 badges earned · preview session



Discover · **See it work** · Predict · Try it · Your challenge

See it work — Card 2 of 5

Watch the story take shape

Read this working example, one line at a time.

```
# Work out the hero's score
gems = 4
coins = 13
score = gems * 10 + coins
print(score)                    1
# Share the coins between 5 friends
print(coins // 5)               2
print(coins % 5)                3
```

What appears on screen

```
53   1
2    2
3    3
```

Python works out gems * 10 first, so the score is 40 + 13. Then // gives each friend's whole coins, and % gives the coins left over.

← Back · One idea at a time. · Continue →

Built for curious minds. Designed for room to grow.

py Python Academy

Courses

Courses › Python Foundations

› Unit 2: Numbers and decisions

Content preview

Jump to any lesson or card. Practice and badges stay in this session; they are not saved.

Return to your account

Lesson 1 of 6 · 0 complete ▾

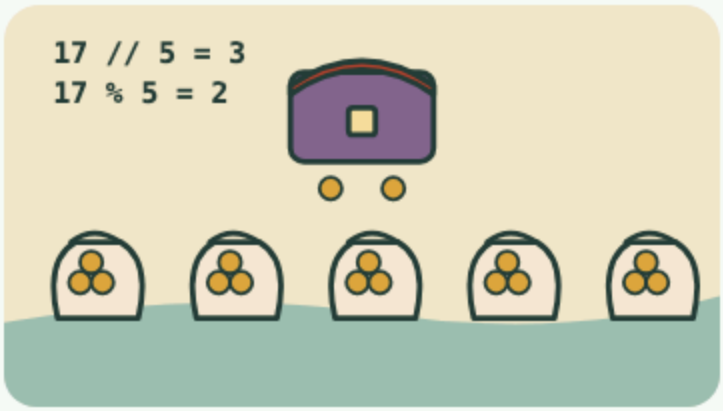
unit 2 · lesson 1 of 6

Count the treasure

`Arithmetic · + - * / // %`

Quest 1 of 6 · Treasure counter

0 XP 0 of 6 badges earned · preview session



Discover · **See it work** · Predict · Try it · Your challenge

See it work

Card 2 of 5

Watch the story take shape

Read this working example, one line at a time.

```
# Work out the hero's score
gems = 4
coins = 13
score = gems * 10 + coins
print(score)                    1
# Share the coins between 5
friends
print(coins // 5)               2
print(coins % 5)                3
```

What appears on screen

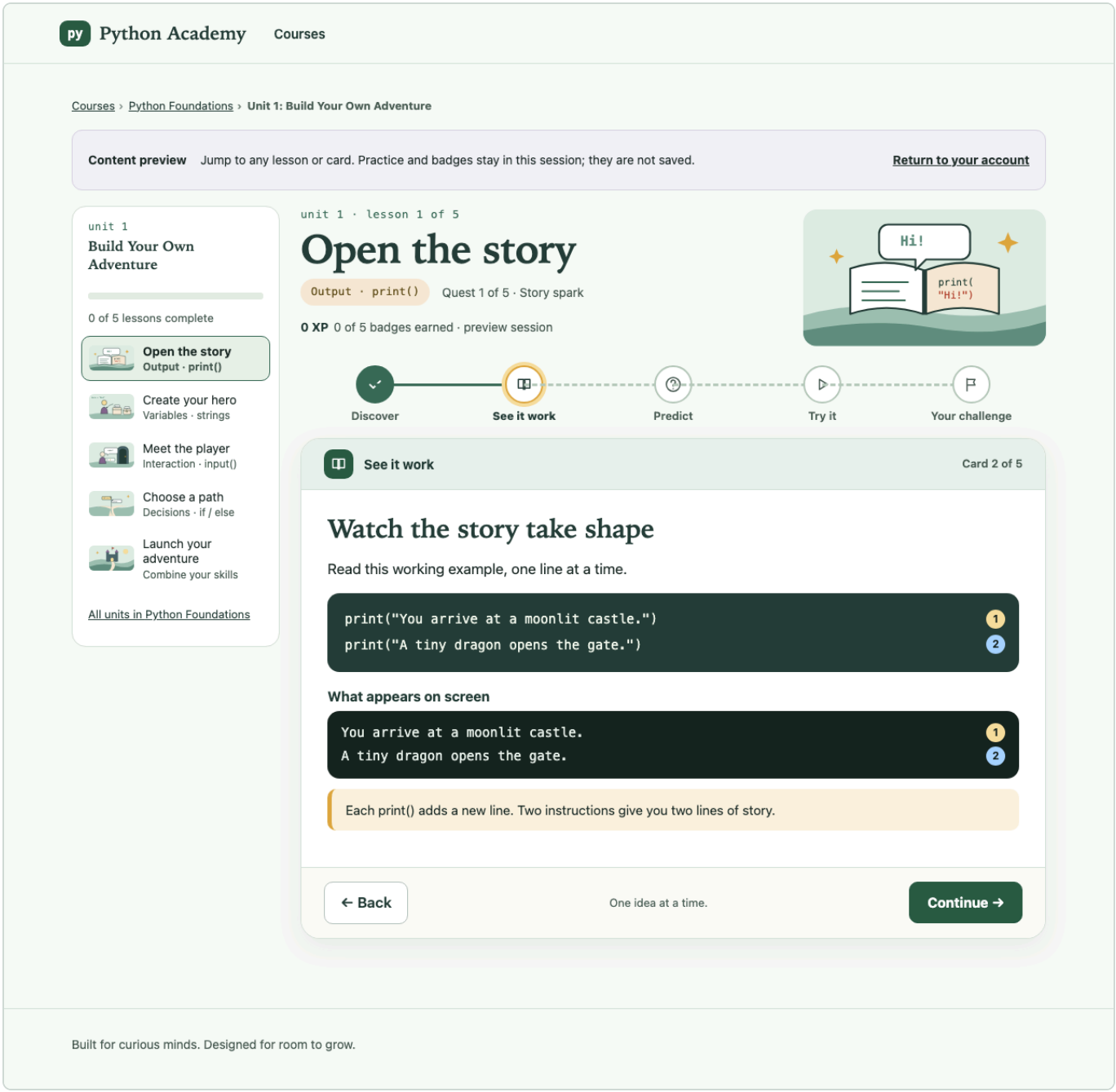
```
53                              1
2                               2
3                               3
```

Python works out gems * 10 first, so the score is 40 + 13. Then // gives each friend's whole coins, and % gives the coins left over.

← Back

Continue →

Built for curious minds. Designed for room to grow.


Python Academy
Courses
Courses › Python Foundations › Unit 1: Build Your Own Adventure
Content preview
Jump to any lesson or card. Practice and badges stay in this session; they are not saved.
Return to your account
unit 1
Build Your Own Adventure
0 of 5 lessons complete
Open the story
Output · print()
Create your hero
Variables · strings
Meet the player
Interaction · input()
Choose a path
Decisions · if / else
Launch your adventure
Combine your skills
All units in Python Foundations
unit 1 · lesson 1 of 5
Open the story
Output · print()
Quest 1 of 5 · Story spark
0 XP 0 of 5 badges earned · preview session
Hi!
print("Hi!")
Discover
See it work
Predict
Try it
Your challenge
See it work
Card 2 of 5
Watch the story take shape
Read this working example, one line at a time.
print("You arrive at a moonlit castle.")
print("A tiny dragon opens the gate.")
What appears on screen
You arrive at a moonlit castle.
A tiny dragon opens the gate.
Each print() adds a new line. Two instructions give you two lines of story.
← Back
One idea at a time.
Continue →
Built for curious minds. Designed for room to grow.

Python Academy Courses

Courses › Python Foundations › Unit 2: Numbers and decisions

Content preview Jump to any lesson or card. Practice and badges stay in this session; they are not saved. Return to your account

unit 2
Numbers and decisions

0 of 6 lessons complete

- **Count the treasure** Arithmetic · + - * / // %
- Average the scores Numbers · int and float
- Score the riddle Conversion · int(input())
- Too high or too low? Comparisons · < > <= >= == !=
- Close enough? Logic · and, or, not
- The riddle keeper's verdict Decisions · elif

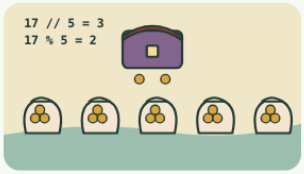
All units in Python Foundations

unit 2 · lesson 1 of 6

Count the treasure

`Arithmetic · + - * / // %` Quest 1 of 6 · Treasure counter

0 XP 0 of 6 badges earned · preview session



Discover · See it work · **Predict** · Try it · Your challenge

Predict Card 3 of 5

What do you think happens?

```
# Work out the hero's score
gems = 4
coins = 13
score = gems * 10 + coins
print(score)
# Share the coins between 5 friends
print(coins // 5)
print(coins % 5)
```

If coins were 14 instead of 13, what would print(coins % 5) show?

A 4

B 2

C 2.8

Let's look again. Five friends take 2 coins each, which uses 10 of the 14. % shows what is left over, so it prints 4. 14 // 5 would show 2, and 14 / 5 would show 2.8.

← Back · One idea at a time. · Continue →

Built for curious minds. Designed for room to grow.

Python Academy Courses

Courses › Python Foundations › Unit 2: Numbers and decisions

Content preview Jump to any lesson or card. Practice and badges stay in this session; they are not saved. Return to your account

unit 2
Numbers and decisions

0 of 6 lessons complete

- **Count the treasure** — Arithmetic · + - * / // %
- Average the scores — Numbers · int and float
- Score the riddle — Conversion · int(input())
- Too high or too low? — Comparisons · < > <= >= == !=
- Close enough? — Logic · and, or, not
- The riddle keeper's verdict — Decisions · elif

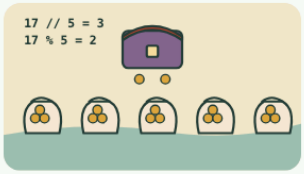
All units in Python Foundations

unit 2 · lesson 1 of 6

Count the treasure

`Arithmetic · + - * / // %` Quest 1 of 6 · Treasure counter

0 XP 0 of 6 badges earned · preview session



Discover · See it work · Predict · Try it · Your challenge

Your challenge Card 5 of 5

Make it your own

The party opens a chest of coins to share. The starter asks for the number of coins and the number of friends. Print two lines: how many whole coins each friend gets, then how many coins are left over. Print just the two numbers. The check runs your program with several chests, including some you can't see.

DONE WHEN YOUR PROGRAM GIVES

- [ ] 17 coins and 5 friends → 3 / 2
- [ ] 20 coins and 4 friends → 5 / 0
- [ ] 3 coins and 5 friends → 0 / 3
- 2 more cases you can't see

Complete the challenge to earn the Treasure counter badge and 100 XP.

Hints are beside your code. Take them one at a time, only if you want them.

story.py Python playground

Your Python code

```
1  # These two lines ask for whole numbe
2  coins = int(input("How many coins? ")
3  friends = int(input("How many friends
4  # Print each friend's whole share, th
5
```

Tab inserts four spaces. To leave the editor, press Esc and then Tab, or Shift + Tab to go back.

This challenge runs your program with its own replies, including some you cannot see, and checks what it prints.

Run & check challenge Reset code

Output

```
Your story will appear here.
```

Hints

Show a nudge

Python runs in your browser. Challenge checks cover the syntax taught in this chapter; badges mark practice, not mastery.

← Back Your story. Your pace. Next quest →

Built for curious minds. Designed for room to grow.

py Python Academy

Courses

Courses › Python Foundations

› Unit 2: Numbers and decisions

Content preview

Jump to any lesson or card. Practice and badges stay in this session; they are not saved.

Return to your account

Lesson 1 of 6 · 0 complete ▾

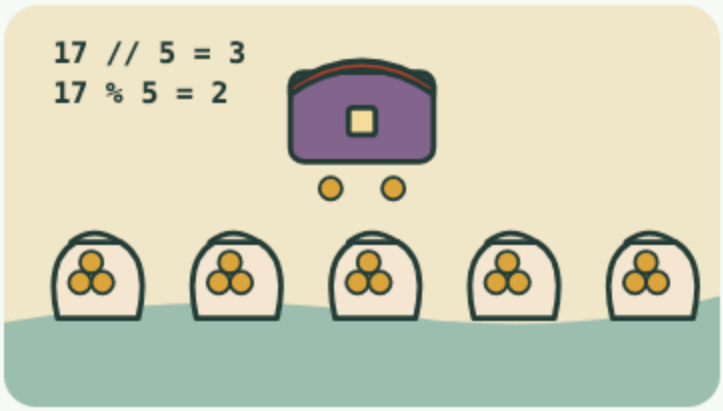
unit 2 · lesson 1 of 6

Count the treasure

`Arithmetic · + - * / // %`

Quest 1 of 6 · Treasure counter

0 XP 0 of 6 badges earned · preview session



Discover · See it work · Predict · Try it · Your challenge

Your challenge

Card 5 of 5

Make it your own

The party opens a chest of coins to share. The starter asks for the number of coins and the number of friends. Print two lines: how many whole coins each friend gets, then how many coins are left over. Print just the two numbers. The check runs your program with several chests, including some you can't see.

DONE WHEN YOUR PROGRAM GIVES

- [ ] 17 coins and 5 friends

```
3
2
```

- [ ] 20 coins and 4 friends

```
5
0
```

- [ ] 3 coins and 5 friends

```
0
3
```

2 more cases you can't see

Complete the challenge to earn the Treasure counter badge and 100 XP.

Hints are beside your code. Take them one at a time, only if you want them.

story.py Python playground

Your Python code

```
1  # These two lines ask for whole
2  coins = int(input("How many co
3  friends = int(input("How many
4  # Print each friend's whole sh
5
```

Tab inserts four spaces. To leave the editor, press Esc and then Tab, or Shift + Tab to go back.

> This challenge runs your program with its own replies, including some you cannot see, and checks what it prints.

Run & check challenge

Reset code

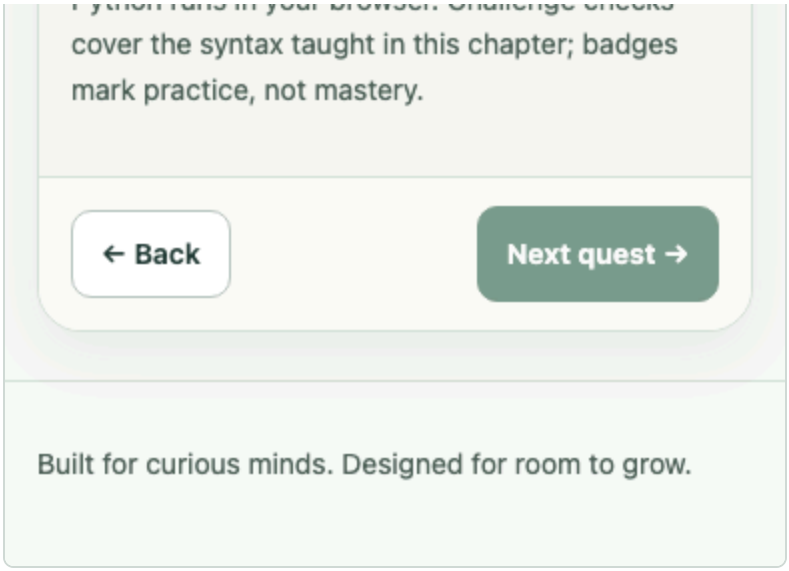
Output

```
Your story will appear here.
```

Hints

Show a nudge

Python runs in your browser. Challenge checks



3. Account rules on the home page

Checked against `backend/app/main.py` `enroll()`, `docs/plan/independent-learners.md` and decision-log chapter 20:

Claim	Source	Result
Invited learners 13 and up; parents over 25	`enroll()` refuses under 13 with an invitation, 25 or under without one	True today
Accounts of their own for learners 16 and up	ID-01, decided 2026-10-27; **ID-01d not built**	Worded as "planned, not open yet"
Invitations expire after seven days and work only for the invited email	`invite()` sets 7 days; `enroll()` checks the email	True
Sign-in is through Google; we never see a password	Google OAuth only	True
Learner code runs in the browser, never on our servers	Pyodide worker; the API compares reports and checks text only	True. Kept because it is in your revised prototype; it is a safety fact, not a "nothing to install" line
Parents see the goal, finished lessons, and attempts	`/api/family` returns each attempt's `passed`, but the page showed only a count	Prototype said "whether each attempt passed". **Changed** to "how many of their attempts passed", and the parent view now shows "N practice attempts, M passed"
"Nothing from a preview is saved"	`/api/check` saves no attempt	True

The line under the sign-in buttons now reads: "Learners 13 and up sign in through a parent's invitation. Anyone over 25 can start a family account. Accounts for learners 16 and up on their own are planned, not open yet."

Could not confirm: whether anyone with a Google account can already sign in today (SEC-10, still open). If they can, a 16 to 25 year old who signs in gets refused at enrolment, as before; the page doesn't promise otherwise.

4. Where I departed from the prototype, and why

Change	Reason
No prototype switcher bar; no "Python in your browser" line	As asked
Playground labelled "Preview", worked out in JavaScript with Python's rules for `//` and `%` (so `-7 // 5` is `-2`)	As asked; it never loads Pyodide, and a test checks no Python file is requested
Lesson stage names kept: Discover, **See it work**, Predict, Try it, **Your challenge**	The prototype says "Worked example" and "Challenge". Kept the existing names so lesson wording and tests stay the same. A one-line change if you prefer the prototype's
Card headings kept ("Watch the story take shape"); the prototype's per-lesson headings ("Share a chest of coins") aren't in the content	No content change in this PR
Discover card has no operator table or callout	Content has only `learn` and `detail` text
The student home keeps an h1 ("Welcome back. Keep building.")	Every page needs one heading; the prototype had none there
The parent and pending pages got simple headings ("Your family space.")	Not in the prototype; the old "Make something with Python" heading is gone
On phones the header's "Courses" link wraps to a second row instead of hiding	Hiding it left no route to the catalog on a phone
Expected output on the ticket is one line per output line, not "3, 2"	Commas are ambiguous when output contains commas
The home page's unit cards and counts come from the catalog	So "three units open, nine on the way" stays true when unit 4 ships. The prototype's descriptions are used for units 1 to 3

5. Tests

Command	Result
`npx tsc --noEmit` and `npm run build`	OK

Command	Result
`CI=1 npx playwright test tests/redesign.spec.ts` (new)	**12 passed**
`CI=1 npx playwright test` (the full browser suite, locally, after the last change)	**374 passed**, including the re-recorded unit 1 snapshot. Earlier runs had one keyboard-test timeout that passed on rerun
Backend	Not changed. Runs in CI

5.1 New tests (`frontend/tests/redesign.spec.ts`)

- The course page's 16 rows each show the picture written out in the test for that lesson id, and each is a real drawing; three lesson headings show theirs, with a description.
- A lesson with no picture (the test fixture lesson) shows the fallback, with its description.
- The ticket shows exactly the plan's visible cases (3 for `py.arithmetic`, 4 for `py.compare`, 3 for `py.while-loops`) and "2 more cases you can't see", and no hidden case's description.
- A doctored plan with a hidden expected output: the page never shows it.
- Unit 1's challenge has no ticket and asks for no plan.
- Markers pair `print(score)`, `print(coins // 5)` and `print(coins % 5)` with `53`, `2`, `3`, and the text stays exact; a loop gets none.
- The playground: `3/2`, Python's negative rules, the zero message, the whole-number message, and no Pyodide, worker or check-runner request.
- The sign-in line's exact words; no "Design prototype" text.
- No sideways scroll at 390 and 1280 px on six pages; cards don't animate with reduced motion.

5.2 Deliberate-break controls (each undone; files matched saved copies byte for byte afterwards)

What I broke	Tests that failed
B1. Two lessons' pictures swapped in `LESSON_ART`	**1**
B2. No fallback for an unknown lesson	**1**
B3. The ticket shows hidden cases as visible	**4**

5.3 Existing tests I changed (intended copy or layout changes only)

Test	Change
`academy.spec.ts` (2), `accessibility.spec.ts` (1)	"Continue with Google" is now "Sign in with Google" (prototype copy)
`academy.spec.ts`, course page	Row state "Completed" is now "Done" (prototype copy)
`academy.spec.ts`, student home (2)	"2 of 16 lessons complete" and "Recommended" are on "Your path in Python Foundations", not an "All courses" card

Test	Change
`lesson-units.spec.ts` (2), `unit2.spec.ts`, `unit3.spec.ts`	The page heading is now the lesson's title; the unit's title heads the lesson list and names the region. The unit summary is no longer on the lesson page
`fixtures/unit1-page.snapshot.json`	Re-recorded. I compared every line of the old and new snapshot (55 views): **every removed line is page chrome** (the old unit heading and tagline, "1. Discover"-style stage labels, the "DISCOVER · Output · print()" eyebrows, the old number marks). **No lesson, hint, check or result line changed.** Added lines are the new heading, path, the words drawn in the pictures, and a screen-reader sentence about markers

One real fault caught by an existing test: my first stylesheet gave every button a hover transition, which broke the rule that the hint nudge never animates (`hints.spec.ts`). Fixed: plain buttons no longer transition.

The full browser suite locally: 374 passed in one run. It has not always been reliable on this Mac, so CI on this PR is the authoritative full run.

6. Accessibility and the rules

- **No new dependency; no inline `style`; no CSP change.** Pictures use classes only.
- **Contrast:** text colours are the prototype's, which meet AA on their backgrounds. The existing axe checks (`accessibility.spec.ts`, WCAG A and AA on the home pages, course page and every lesson card, desktop and phone) pass.
- **Keyboard:** every control keeps a visible 3 px focus ring; the path buttons and predict tiles are buttons; `keyboard.spec.ts` passes.
- **Screen readers:** pictures beside their own title are hidden; the others have a description. Predict letters are hidden so each answer is announced by its text. The path's buttons are announced "1. Discover" and so on. Ticket states are in words.
- **Reduced motion:** card and badge animations and transitions only apply under `prefers-reduced-motion: no-preference`.

7. Visual track overlap

Other stages that only change looks stay blocked on this PR, as asked:

Stage	Overlap	Recommendation
SITE-03 (product polish: tokens, course header art, lesson celebration)	This PR does the tokens, the pictures and most of the polish	Rescope SITE-03 to what's left (course emblems, a celebration) after this merges

Stage	Overlap	Recommendation
SITE-02 (separate public story site)	The new signed-out home already works as a landing page	Decide whether SITE-02 is still wanted, or whether this home page is enough for now
SITE-01 rules (`site-design.md` section 2)	It says "no text inside images" and "soft gradients". Your prototype's pictures contain code text and are flat	I followed the prototype. If you agree, update `site-design.md` in the next docs sync
UI-01 (cosmetic fixes, merged)	Its tests (chips on one line, goal text size) still pass	None
UI-02 to UI-04 (wait, decision 29)	Not visual, but they would change the student home, now `home.tsx`	Build on this PR's `StudentHome`
WE6 / QA-03 (accessibility, cross-browser)	The weekly cross-browser run will now cover the new pages	Watch the first WebKit run after merge (4 known WebKit failures before this PR)

I did not edit `backlog.md` or the decision log, as asked; the next docs sync should add this PR.

8. What I could not verify

- **Real devices and screen readers.** Checked in Chromium with Playwright at 390 and 1280 px only; not in Safari, Firefox, on a real phone, or with VoiceOver or NVDA.
- **The live site.** Not deployed.
- **Fonts:** serif headings use the system serif stack (Iowan Old Style, Palatino, Georgia). How they look on Windows and Android wasn't seen.
- **"returns" values on the ticket** are shown in Python style from JSON, so a whole-number float such as `2.0` would show as `2`. No released lesson's visible case has one today.
- **The footer** says "Three units are open": prototype copy, written by hand. Update it when unit 4 ships.
- **Pictures in the 52 px unit list** are decoration at that size; their code text is unreadable there (the lesson title sits beside them).

9. Questions for you

1. **Stage names:** keep "See it work" and "Your challenge", or switch to the prototype's "Worked example" and "Challenge"? *Recommended: the prototype's, in a small follow-up that also updates the lesson tests.*
2. **Text in pictures:** accept code text inside the lesson pictures, against `site-design.md`'s rule? *Recommended: yes for code, which is the idea being taught, but never sentences.*
3. **Parents' fact line:** is "how many of their attempts passed" right, or do you want the parent view to list each attempt?

10. Live checks for you (after merge and deploy)

1. Open the home page signed out, on a phone and a computer. Change the playground's numbers; check DevTools → Network shows no `pyodide` or `python-worker.js` request.
2. Sign in as a student: the home shows your next lesson with its picture.
3. Open a unit 2 challenge: the ticket lists the cases; run a wrong answer and see a case marked "not yet".
4. DevTools → Console shows no CSP reports on any page.

11. Next concrete action

Review the screenshots and merge once CI is green.

12. Removing the worktree safely

From the **main** folder, after this PR is merged:

```
cd /Users/DEV/Projects/python-academy
git -C ../python-academy-redesign status --short   # expect nothing
git worktree remove ../python-academy-redesign
git worktree prune
git branch -d ui/visual-redesign
```